PUBLIC TECHNICAL WHITEPAPER

ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

Security evolution · Threat verification · Cross-platform evidence

ENGLISH EDITION

September 2026

DOCUMENT MAP

Contents

Product overview and architecture are maintained separately. This document focuses on security evolution and executable evidence.

This document does not cover the product overview or product architecture. Those subjects are addressed separately in ODRE PQC v0.2.9 Product Overview and Security Architecture. This whitepaper records the development and security evolution of ODRE PQC, threat-model verification, platform-specific execution results, and the traceability of sealed artifacts to supporting evidence.

1. Development and Security Evolution

1.1 v0.1.1: Blocking Plaintext Bypass on Protected Routes

The v0.1.1 report documents the remediation of a protected-route plaintext bypass classified as PQC-001 HIGH in the preceding audit. If a protected route can reach a consumer handler after the security runtime fails, application behavior may execute even though the cryptographic boundary has failed.

v0.1.1 made /secure/* fail closed and blocked handler bypass after security failure. Verification of the v0.1.1 wheel recorded:

- Wheel SHA-256: 627AF68632D5C99481849CB1C43497381267CCE426F3A5DA9752B069C36D37E6
- RECORD integrity: 20/20
- Import safety verified
- install(app) plug-in API verified
- Protected-route policy: fail closed
- Consumer-handler executions after security failure: 0
- Isolation of public and protected routes verified
- Resource lifecycle and cleanup verified

This change became the baseline security contract for later versions. Subsequent releases expanded verification beyond repetition of the same tests to replay, request binding, native-runtime trust, crash recovery, and platform-specific attacks.

1.2 v0.1.2: Three Defects Found by Clean-room Attack Testing After Functional Verification

Two reports with different purposes remain for v0.1.2. The product verification report examined the installation API, fail-closed routes, rotation policy, transactional security-database migration, shutdown draining, diagnostic commands, and client round trips. It recorded 38/38 overall tests, 37/37 configuration and rotation misuse scenarios, and ten startup/shutdown cycles. The wheel SHA-256 was 59A562C789B28C05B309AAD0BC51511D9BD16D84A783021F1598A8AF21B4F06D.

An independent clean-room re-verification installed the same wheel in a new Python 3.12 environment with hash-locked dependencies and passed only 17 of 20 security attack tests. Installation, the real-PQC happy path, client round trips, RECORD 30/30, and resource lifecycle passed, but three security defects were reproduced. Functional and regression success therefore could not be extended to the entire attack surface.

1.2.1 F-001: Sequence Gap Accepted

The test withheld sequence 5 and sent sequence 6. The expected behavior was rejection of the gapped request without protected-handler execution. Instead, sequence 6 was accepted with HTTP 200 and the protected-handler count increased from 3 to 4.

The issue was continuity, not merely duplicate replay. If a system rejects duplicates but accepts any value greater than the current sequence, an attacker can skip an intermediate request or reorder traffic while still advancing the session. Sequence verification must atomically establish that a value is exactly the next value, not simply unused.

1.2.2 F-002: Handler Executed Before Oversized-response Decision

In a test where the protected handler generated an oversized response, the external response failed closed with HTTP 503 and no plaintext leak was observed. The handler nevertheless executed once before the size decision.

The final network response is not the only security concern. A handler can modify a database, call an external service, or write a file. Rejecting the output after execution may still permit application side effects. Unsupported response contracts must therefore be identified before handler entry.

1.2.3 F-003: Handler Executed Before Unsupported-streaming Decision

Streaming, SSE, iterator, and WebSocket responses were unsupported by the protected-response contract at that time. The external response failed closed with HTTP 503 and did not expose plaintext, but the handler executed once before support was evaluated. As with F-002, the fail-closed decision occurred too late.

The v0.1.2 clean-room report preserved these findings without modifying the product during the audit. Remediation and closure were assigned to v0.1.3.

1.3 v0.1.3: Strict Sequence and Pre-handler Contract Enforcement

v0.1.3 closed all three v0.1.2 findings by changing the order of enforcement at the security boundary.

- Sequence gaps, rollbacks, and duplicates were all rejected.
- Oversized or unbounded protected-response contracts were rejected before handler execution.
- Unsupported streaming, SSE, iterator, and WebSocket types were rejected before handler execution.

Clean-room re-verification installed the wheel with --require-hashes in a new Python 3.12 environment and executed the installed copy from site-packages. The happy path used real OpenSSL 3.5.8 ML-KEM-768 and ML-DSA-65 operations rather than mock cryptography.

Verification area	Observed result
Security attack tests	20/20
Handler executions after security failure	0
Misconfiguration tests	6/6
Wheel RECORD	31/31
New Critical / High / Medium findings	0/0/0

Gap, rollback, and duplicate requests were rejected with zero protected-handler executions. Oversized, unbounded, and unsupported streaming responses were rejected before the handler and no plaintext leak was observed. The audited wheel SHA-256 was EAA164895F835478441465112AAEEF8F6768740FFE2908BBE7CD53298D934D12.

1.4 v0.1.4: Observability Added Without Regressing the Security Contract

v0.1.4 added status observability and verified that the existing protection boundary was not weakened. The status feature passed 15 product tests and four independent live-runtime checks. The unchanged v0.1.3 security suite passed 41/41; the combined development result was 56/56.

Clean-room re-verification covered:

- New Python 3.12 environment with hash-locked dependencies
- Execution from the installed wheel
- Real OpenSSL runtime
- Real ML-KEM-768 and ML-DSA-65 happy path
- Security attack tests: 20/20
- Misconfiguration tests: 6/6
- Handler bypasses: 0
- RECORD: 32/32
- State mutations caused by status queries: 0
- Secret leakage in status output: 0
- Automatic classical downgrades: 0
- New Critical / High / Medium findings: 0/0/0

The wheel SHA-256 was 19EF222083C15D89E63B49A08D62EFEF480F3B82443839C900B6DAE425B13E54. v0.1.4 did not test only the new feature; it reconfirmed in a later version the sequence and pre-handler conditions that had failed in v0.1.2.

1.5 v0.1.2-v0.1.4 Improvement Lineage

Version	Finding or change	Attack or regression risk	Structural correction	Re-verification
v0.1.2	Sequence gap accepted	Session advances after request-order disruption	Need for strict next-sequence check established	Clean-room 17/20; failure preserved
v0.1.2	Handler before oversized-response decision	Application side effect before external rejection	Pre-handler response-contract evaluation	v0.1.3 handler executions: 0
v0.1.2	Handler before unsupported-streaming decision	Side effects from an unsupported type	Pre-handler rejection of unsupported types	v0.1.3 handler executions: 0

Version	Finding or change	Attack or regression risk	Structural correction	Re-verification
v0.1.3	Three findings remediated	Fix may break happy path or packaging	Strict sequence and pre-handler enforcement	Attack 20/20; configuration 6/6; RECORD 31/31
v0.1.4	Status observability added	Query mutates state or leaks secrets	Read-only, secret-free status contract	Status 15+4; prior security 41/41; RECORD 32/32

1.6 v0.2.4 to v0.2.5: Ubuntu Native ABI Defect

When v0.2.4, which had passed Windows verification, was tested in a real Ubuntu environment, the Linux shim was found to omit the odre_shutdown export. The case demonstrated that native-runtime ABIs require independent platform verification even when the Python integration surface is identical.

v0.2.5 corrected and verified:

- Expected Ubuntu-shim exports: 15; actual exports: 15
- odre_shutdown export present
- Double shutdown and startup-shutdown-startup lifecycle
- Five of five repeated cycles
- File-descriptor delta: 0
- Shim unmapped after shutdown
- OpenSSL 3.5.8 runtime identity
- Real ML-KEM-768 and ML-DSA-65 execution

A targeted v0.2.5 re-audit subsequently found one additional High and two Medium findings, so release blocking remained in place. Their root causes and remediation belong to the v0.2.6 lineage.

1.7 v0.2.6: Native Trust, TOCTOU, and FD Identity Hardening

The v0.2.6 evidence is separated into blind static audit, targeted reproduction, confirmed root cause, static-to-dynamic classification, remediation, and final multi-review audit. Static suspicions were not automatically labeled as defects; each was exercised with an attack harness and classified by reproducibility.

The principal hardening areas were:

- Closing TOCTOU between the object hashed and the object loaded
- Binding trust to actual file-descriptor and inode identity rather than a path string
- Validating native-provider and dependency-search paths
- Proving verified-object and loaded-object identity during replacement, reuse, and path-switch attacks
- Re-running regression tests on both platforms after remediation

The official v0.2.6 wheel SHA-256 was 4756FC9B219DAB4E2BF9C90FA066CFFD0D5DEA94FEF03DCE0B18DED55E5A7C7A.

1.8 v0.2.8 to v0.2.9: F-028-01 and Parent-object Replacement Defense

F-028-01 was identified in v0.2.8. The vulnerable initialization path could allow a pre-promotion Path.resolve() result to be influenced by a transient root symlink.

On Linux, v0.2.9 opens and retains the trusted parent-directory chain with O_DIRECTORY | O_NOFOLLOW, locks staging-directory identity, and promotes it using parent-dirfd-relative renameat2(..., RENAME_NOREPLACE). It then verifies that the promoted directory (st_dev, st_ino) matches the staging object. Initialization fails closed if the required primitives or identity proof are unavailable.

Targeted attack results:

- Trials: 600
- Symlink swaps: 1,212,203
- Exit distribution: 278 successes / 322 safe aborts
- Object-path misbindings: 0
- External-path persistence: 0
- External-file mutations: 0
- Secret leakage: 0
- Partial promoted state: 0

The subsequent full audit ran a separate 600-trial late-promotion parent-object attack. Initialization failed closed in all 600 attack states, with zero object-path misbindings and zero external secret state.

2. Threat Model and Security Verification

The verification scope included:

1. Replay and sequence races
2. Request and response tampering
3. Cross-session and cross-path attacks
4. Malformed and oversized input
5. HTTP-smuggling shapes
6. Fuzz, soak, and load
7. TOCTOU, file-descriptor, and inode identity
8. Symlink, junction, and reparse attacks
9. Loaded-object identity
10. Provider, dependency, and configuration injection
11. Worker collision and concurrent initialization
12. Crash, restart, and shutdown lifecycle

13. Stale session, lease, and status reuse

2.1 Replay and Sequence Races: Final Observations

Platform	Duplicate requests	Acceptances per sequence	Double handler executions
Windows Server 2022	1,280	Exactly one	0
Ubuntu 24.04 LTS	2,880	Exactly one	0

Replay testing did not rely only on whether an error response appeared. It separately instrumented whether concurrent delivery of the same sequence caused the application handler to execute twice.

2.2 Native Loaded-object Identity: Final Observations

On Windows, the tests observed 3,000 actual provider/WinDLL loads, 1,062 successful atomic replacements, and 3,000 junction create/remove operations. Untrusted DLL loads, untrusted dependency loads, reparse bypasses, and verified-versus-loaded identity mismatches were all zero.

Ubuntu verification covered atomic file, parent, and symlink swaps; unsafe parents; dependency, provider, module, engine, and configuration injection; FD/inode identity; and 50+50 lifecycle cycles.

3. Platform-specific Verification

3.1 Windows Server 2022

- Windows Server 2022 AMD64
- CPython 3.12.10
- Hash-locked offline installation
- init, second-init protection, doctor, verify, and status
- Real ML-KEM-768, ML-DSA-65, and hybrid handshake
- Request semantic binding and response metadata boundary
- ACL trust, status authentication, and shutdown resource lifetime
- Forced termination and second-crash recovery

3.2 Ubuntu 24.04 LTS

- Ubuntu 24.04.4 LTS AMD64
- CPython 3.12.3
- Hash-locked offline installation
- init, second init, doctor, negative doctor, verify, and status
- Real ML-KEM-768, ML-DSA-65, and hybrid handshake
- Admission fuzz and malformed or oversized input
- HTTP-smuggling shapes and WebSocket fail-closed behavior

- Native trust, symlink and parent swaps, and FD/inode identity
- Graceful shutdown, restart, SIGKILL, and crash recovery

4. Artifacts and Evidence

4.1 Sealed Wheel

- File: odre_pqc-0.2.9-py3-none-any.whl
- SHA-256: A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
- RECORD: 45/45 hashed entries verified
- Package and runtime versions matched at 0.2.9
- Audit-start and audit-end SHA-256 values identical

4.2 Native Runtime Archives

- Windows runtime SHA-256: 826303E72A76B10041D26113F465ADD8960825C71F50722D6792174E73F9D12C
- Ubuntu runtime SHA-256: EFAD16698203371D07F25B429F6731B9C0954EEBF5B58A0024789EF0611A78D1

4.3 Evidence-report Mapping

Whitepaper area	Principal evidence reports
v0.1.1	ODRE PQC v0.1.1 - Independent Audit Evidence Log
v0.1.2-v0.1.4	Version and clean-room re-verification reports
v0.2.0	v0.2.0_FINAL_REPORT.md
v0.2.1	v0.2.1_CUSTOMER_ZERO_CONFIG_BOOTSTRAP_REPORT.md, v0.2.1_FINAL_AUDIT_REPORT.md
v0.2.5	v0.2.5_FINAL_REPORT.md, v0.2.5_REMEDIATION_HISTORY.md
v0.2.6	Blind static, targeted reproduction, root-cause, remediation, and multi-review reports
v0.2.7	Independent audit and remediation reports
v0.2.8	F-027-01, RC03, F-028-01, and final release-audit reports
v0.2.9	F-028-01 targeted remediation and full development-security audit

4.4 Separating Harness Problems from Product Defects

Six harness problems identified during the final audit were preserved separately from product defects. After fixture and binding corrections, the relevant gates were re-executed without changing product bytes. Harness errors were not automatically interpreted as product success or failure; the corrected and re-executed evidence was retained with the results.

5. Conclusion

ODRE PQC security improvement was an iterative process that preserved failed tests and connected their attack causes to structural changes in later versions, rather than a single pass event. v0.1.1 blocked protected-route plaintext bypass. The v0.1.2 clean-room attack suite exposed sequence-gap acceptance and handler-ordering defects. v0.1.3 closed them with strict sequence and pre-handler contract enforcement, while v0.1.4 added observability and reconfirmed the prior security contract.

Later releases expanded verification from the Python integration boundary to native runtime and operating-system object trust. Ubuntu ABI omission, TOCTOU between verification and loading, FD/inode identity, symlink/junction/reparse attacks, provider/dependency/configuration injection, and crash/restart/shutdown lifecycle were treated as distinct attack conditions. The sealed v0.2.9 wheel was re-verified on Windows Server 2022 and Ubuntu 24.04 LTS while retaining the same SHA-256.

The final identified artifact is odre_pqc-0.2.9-py3-none-any.whl, SHA-256 A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697. Supporting evidence links RECORD 45/45, audit-start/end artifact identity, the real OpenSSL 3.5.8 runtime, ML-KEM-768, ML-DSA-65, hybrid handshake, replay races, native loaded-object identity, and lifecycle tests to that artifact.